

FiSH General Ledger Engine

Requirements & Specification Document

Prepared for: Femi Falase

Document date: August 11, 2026

Status: Draft v0.7

Document Control

Version	Date	Author	Summary
0.1	2026-08-11	Claude (drafted with Femi)	Initial generic draft, written before business plans were available
0.2	2026-08-11	Claude (drafted with Femi)	Full rewrite grounded in Purse/Scrip/Osusu business plans, ethical framework, and the existing Kotlin domain model
0.3	2026-08-11	Claude (drafted with Femi)	Added FiSH's third customer segment (B2B ledger platform for any company, Mano River go-to-market) and government tax-due computation/reporting as a functional requirement
0.4	2026-08-11	Claude (drafted with Femi)	Reframed prohibited-purpose screening as a partner/borrower affirmation rather than FiSH-performed sector classification
0.5	2026-08-11	Claude (drafted with Femi)	Added company/business debt resolution via capitalization into equity and turnaround (business continuity as prime focus), distinct from individual Jubilee forgiveness, with a Scrip-managed EquityStake entity
0.6	2026-08-11	Claude (drafted with Femi)	Added default, real-time business-health signal (Growing/Stagnant/Regressing/Dying) as a Scrip treasury-management service, with the GL Engine limited to supplying near-real-time trend data/event notifications
0.7	2026-08-11	Claude (drafted with Femi)	Named BuzzMe — a fourth concept (Mano River e-money/mobile-money payment system on Christian ethics) — as not-yet-scoped, alongside Purse/Scrip/Osusu

1. Executive Summary

FiSH (Financial [Information] Systems Handler) is the shared software platform — package `com.theprodeogroup.fish` — underlying a family of related financial ventures Femi is building. Its core is the General Ledger (GL) Engine: a multi-tenant, multi-user, multi-company, double-entry accounting system with a full audit trail, built to serve as the system of record for every other component.

Three ventures sit on top of it:

- **Purse** — a Christian credit union, launching in the UK (FCA/PRA-regulated) and expanding to Northern Ireland, the Republic of Ireland, and Sierra Leone.
- **Scrip** — Purse's treasury/investment arm, kept legally distinct, being built in Rust.
- **Osusu** — a rotating-savings and social-graph micro-lending product run through Purse, being built in Kotlin, with a distinct go-to-market in Sierra Leone's agrifinance sector.

Beyond these three, FiSH is also offered directly as a B2B ledger/SaaS platform to any company that wants to process its financial information through it, with a specific go-to-market emphasis on companies in the Mano River region (Sierra Leone, Liberia, Guinea, Côte d'Ivoire). Because the engine holds a company's full, audited transaction history, it can also compute taxes due and generate government-ready tax reports directly from that ledger data — turning routine bookkeeping into a byproduct of accurate tax reporting. A fourth concept, BuzzMe — an OPay/PalmPay-style e-money payment system for the Mano River region, built on Christian ethics — is named but not yet scoped (Section 2.7).

Development of the GL Engine's domain model has already started (Kotlin, `com.theprodeogroup.fish.domain.common`): audit actions, posting/period lifecycles, journal sources, client types, and transaction sides are coded as enums today. This document specifies the GL Engine's requirements in light of that existing code and the three ventures' business plans, so that architecture and remaining implementation can proceed against a single agreed scope.

2. Business Context

2.1 Purse — Christian Credit Union (UK / NI / ROI)

Purse (trading as “The Scrip and Purse” in early planning materials) is proposed as a UK credit union registered under the Co-operative and Community Benefit Societies Act 2014, dual-regulated by the FCA and PRA, with FSCS deposit protection and FOS complaints handling. It targets the UK's Christian population (an estimated 38 million, with an initial focus on ~6 million active church members across 47,000 churches), offering savings, ethical loans, and later mortgages and business banking, delivered 100% digitally (mobile + web).

Membership requires professing the Christian faith (Apostles' or Nicene Creed) or belonging to a qualifying Christian organisation — this “common bond” is a specific KYC/onboarding requirement the GL Engine's client model must support (see Section 7.11).

Geographic rollout: England first (London, Manchester, Birmingham), then Scotland/Wales/Northern England, then Northern Ireland and the Republic of Ireland (targeted Year 6, via EU passporting).

Lending is governed by an explicit ethical/theological framework (Section 2.5) with hard interest-rate ceilings, prohibited-sector screening, and a structured arrears/forgiveness process — all of which are ledger and audit-trail requirements, not just policy.

2.2 Purse / FiSH GL — Sierra Leone Agrifinance

A parallel, more advanced business plan exists for Sierra Leone (docs/SL/FiSH_GL_Business_Plan.md in this project): FiSH GL as the ledger/credit-scoring engine, with Purse as the consumer-facing product digitising informal Osusu savings and layering in social-graph micro-lending, targeted initially at the rice value chain. Distinguishing features versus the UK plan: SIM Toolkit/USSD access (not smartphone-dependent), a treasury layer with a gold-referenced reserve tranche, dual-track SACCO (Cooperative Societies Act 1977) + Bank of Sierra Leone Sandbox registration, and a B2B credit-scoring-as-a-service revenue line sold to outgrower schemes and commodity buyers.

The GL Engine needs to support both a smartphone/web-first UK credit union and a USSD-first Sierra Leone agrifinance product on the same core ledger.

2.3 Scrip — Treasury / Investment Arm

Scrip is planned as a legally separate entity (Scrip Capital LLP/Ltd) from Purse, providing the trading/investment desk — pooled deposit management, UK Gilts and screened corporate bonds, and (in the Sierra Leone plan) a gold-referenced reserve tranche. It is being written in Rust, as an alternative to C++, and is expected to be a client of the GL Engine's API rather than sharing its codebase.

Scrip's treasury management services are also expected to include, by default (not as a premium add-on), continuous business-health monitoring for the businesses it serves: a real-time signal telling a business owner whether their company is Growing, Stagnant, Regressing, or Dying, derived from that company's GL Engine ledger data (see Section 7.14).

2.4 Osusu

Osusu is the rotating-savings/community-finance product run through Purse: automated thrift savings, social-graph-based micro-lending (“Osusu Advance”), and — in the Sierra Leone context — harvest-cycle agri-lending. Osusu-related code is being written in Kotlin, alongside the rest of the FiSH/Purse platform.

2.5 Ethical & Theological Lending Framework

Purse operates under a written ethical framework with concrete, system-relevant rules:

- **Interest rate ceilings by product:** personal loans 12.68% APR, emergency/short-term 18% APR, secured loans 9% APR, absolute ceiling 26.82% APR — each with a documented rate-breakdown (cost of funds, operating costs, risk premium, capital reserve, admin) shown to the borrower.
- **Prohibited-purpose screening:** the written framework lists prohibited/grey-area sectors, but FiSH implements this at the system level as a partner/borrower affirmation rather than automated sector classification — Purse doesn't have the expertise to categorise every business type, so the ledger instead captures a required, audited declaration that funds will not be used for illegal or unethical purposes (see Section 7.9).
- **Structured arrears process:** four stages (early contact, pastoral engagement, formal review, final resolution) with defined day-ranges, culminating in either a negotiated settlement/partial write-off or formal collection — every stage and outcome needs to be an auditable event.
- **Debt forgiveness (“Year of Jubilee”), individual members:** every 7 years the board reviews long-standing hardship arrears for partial/full forgiveness — this must be a supported, audited transaction type, not a manual workaround.
- **Debt resolution, company/business borrowers:** for a company in hardship-but-recoverable circumstances, “forgiveness” is not a simple write-off — Purse capitalises the outstanding debt

into an equity stake, takes the company into administration with the objective of a turnaround, and holds an option (not obligation) to sell that stake back to the original owner once circumstances make it possible. Business continuity is the prime focus — the goal is keeping the company (and the livelihoods depending on it) operating, not maximising recovery or converting distress into a permanent ownership position. This is fundamentally an investment/restructuring operation, more naturally the domain of Scrip (Section 2.3) than Purse's day-to-day lending ledger (see Section 7.13).

- **Transparency reporting:** annual disclosure of where funds were lent (by category), investment holdings, loan-loss rate, and aggregate interest margin vs. cost — i.e., specific standing reports the GL Engine must be able to produce.

2.6 FiSH as a B2B Ledger Platform — Mano River Region

Independently of Purse/Scrip/Osusu, FiSH is intended to be offered as a standalone multi-tenant ledger/SaaS platform to any company that wants to process its financial information through it. The specific go-to-market emphasis is companies operating in the Mano River region — Sierra Leone, Liberia, Guinea, and Côte d'Ivoire — a natural extension of the agrifinance groundwork already laid in the Sierra Leone business plan (Section 2.2) and of Scrip/Osusu's planned regional reach.

This is a third, distinct customer segment alongside Purse (the credit union) and the general SMB/individual clients the existing ClientType model already supports. Because these companies' full transaction history sits in FiSH, the engine is also positioned to compute taxes due and generate government-ready tax reports directly from posted ledger data (Section 7.12) — a proposition of particular value in economies where informal record-keeping makes tax collection difficult, echoing the underwriting-gap logic already documented for credit in the SL business plan.

2.7 BuzzMe — Mano River Payment System (Concept, Not Yet Scoped)

BuzzMe is the branded name for a possible future e-money/mobile-money payment system for the Mano River Union region (Sierra Leone, Liberia, Guinea, Côte d'Ivoire), functioning like OPay or PalmPay but built on Christian ethics.

This is a fourth named concept alongside Purse, Scrip, and Osusu — flagged here for visibility, not yet spec'd to the level of the rest of this document. What's already understood about it:

- **Different product surface than Purse:** wallet balances, an agent cash-in/cash-out network, merchant QR/POS, P2P transfers, and interoperability with telco mobile money (Orange Money, Africell Money) — not a deposit-taking credit-union account model.
- **Different regulatory track:** e-money/PSP licensing per country, rather than Purse's credit-union/SACCO registration route.
- **Same underlying ledger:** BuzzMe transactions would be expected to settle through the same FiSH GL Engine as Purse, Scrip, and Osusu.
- **Same ethical carry-over:** the screening/rate-ceiling logic already built for Purse (Sections 2.5, 7.9) would be expected to extend to BuzzMe rather than be reinvented.

A dedicated business plan and requirements pass for BuzzMe would be needed before it's added to the functional requirements (Section 7) or data model (Section 10) of this document.

3. Why a Shared GL Engine

Rather than building separate ledgers for the UK credit union, the Sierra Leone agrifinance product, and Scrip's treasury book, FiSH centralises double-entry bookkeeping, chart of accounts, periods, and audit trail in one engine that:

- Scrip (Rust) and Osusu (Kotlin) integrate with as API clients, posting their transactions into it as the system of record.
 - Supports the general-purpose client types already modelled in code (ClientType: Individual, Sole Trader, Partnership, Company Limited, Non-Profit), meaning the same engine can also serve ordinary SMB/individual bookkeeping customers beyond Purse — this generality is already a design decision, not an open question.
 - Enforces jurisdiction- and product-specific rules (interest ceilings, arrears workflow, period locking for regulatory close) as first-class ledger behaviour rather than external policy layered on top.
 - Lets any onboarded company — including the Mano River B2B segment (Section 2.6) — get accurate government tax-due computation for free, simply by keeping its books on FiSH.
-

4. Purpose & Objectives

- Provide accurate double-entry bookkeeping with a configurable chart of accounts, usable both for Purse/Scrip/Osusu and for general SMB/individual clients.
 - Support multiple tenants, multiple users per tenant, and multiple companies per tenant (group structures) — e.g., Purse UK, Purse NI/ROI, and Purse Sierra Leone as distinct companies/entities that may need consolidated or separate reporting.
 - Produce standard financial statements and the specific transparency/regulatory reports Purse's ethical framework and FCA/PRA/BSL regulators require.
 - Maintain a complete, immutable audit trail — already partly modelled via AuditAction — sufficient for SOX/GDPR-style compliance as referenced in the existing code comments.
 - Expose an API so Scrip (Rust) and Osusu (Kotlin) can post transactions and pull reports/balances programmatically, and so a future USSD gateway (Sierra Leone) can post through the same interface.
 - Enforce product-specific business rules (interest rate ceilings, prohibited-purpose flags, arrears stage transitions) at the ledger/domain layer, matching the PostingStatus/PeriodStatus state-machine approach already used in code.
 - Compute taxes due to the relevant government/tax authority directly from a company's posted ledger transactions, and produce government-ready reports — starting with the Mano River launch markets (Sierra Leone, Liberia, Guinea, Côte d'Ivoire).
-

5. Scope

5.1 In Scope (v1)

- Multi-tenant, multi-user, multi-company data model with full tenant isolation.
- Configurable chart of accounts per company, with default templates keyed to ClientType (Individual, Sole Trader, Partnership, Company Limited, Non-Profit) — plus a new template for credit union / cooperative structures (see Section 13, Open Questions).
- Journal entries with full double-entry validation, matching the existing PostingStatus lifecycle (Draft → Pending → Posted → Reversed, with Rejected and System variants) and JournalSource tagging (Manual, API, Integration, Import, System, Reversal, Closing).
- Fiscal period management matching the existing PeriodStatus lifecycle (Draft → Open → Closed → Locked, with reopen from Closed) and configurable PeriodType (Month, Quarter, Year, Custom).
- Multi-currency support (at minimum GBP, EUR, SLE, USD) with FX rate capture and revaluation.
- Full audit trail using the existing AuditAction taxonomy (CRUD, state transitions, period operations, security events, data operations, access tracking).
- Multi-dimensional tagging on transactions using the existing DimensionType enum (Cost Center, Department, Project, Location, Product Line, Customer, Vendor, plus 3 custom slots) — useful for Purse reporting by branch/region and for Osusu reporting by savings circle.
- Role-based permissions, enforced server-side, not just in the UI.
- REST API for Scrip and Osusu to post entries and pull balances/reports.
- Interest-rate-ceiling enforcement at the point of loan-related posting, configurable per product type, matching Purse's documented APR ceilings.
- Loan-purpose affirmation: partner/borrower affirms funds will not be used for illegal or unethical purposes, captured as a required, audited declaration at loan origination — not FiSH performing sector/category screening.
- Arrears workflow as a first-class, staged process (mirroring the four-stage model in the ethical framework) with each stage transition captured in the audit trail.
- Standing reports needed for Purse's annual transparency disclosure (lending by category, loan-loss rate, interest margin vs. cost) and for FCA/PRA-style regulatory reporting.
- Configurable, jurisdiction-specific tax computation (starting with Sierra Leone, Liberia, Guinea, Côte d'Ivoire) that derives taxes due from posted ledger transactions and produces a government-ready tax report per company/period (see Section 7.12).
- Near-real-time exposure of a company's posted transaction data and standard trend metrics (e.g., revenue/expense trajectory, cash flow trend) via API/event notifications, so Scrip's business-health monitoring (Section 7.14) can consume it without waiting for a batch report cycle.
- Export to CSV/PDF; report data also available via API.

5.2 Out of Scope (v1 — candidates for later phases)

- Direct payment rail integration (ClearBank/Modulr, Faster Payments, Pay.UK) — GL Engine should be ready to receive postings from a payments layer, but does not own payment execution.

- AML/KYC identity verification and PEP/sanctions screening (Onfido, World-Check, SAR generation) — GL Engine should be able to record the outcome of these checks and gate postings on them, but does not perform the screening itself.
 - USSD/STK gateway implementation (Sierra Leone access layer) — out of scope for the engine itself; the engine exposes an API the gateway will call.
 - Credit-scoring model/engine (Osusu contribution consistency, social-graph vouching, harvest-cycle cash-flow modelling) — GL Engine supplies the transaction history this model consumes, but does not build the model.
 - Business-health classification/scoring engine (the logic that turns trend data into a Growing/Stagnant/Regressing/Dying determination) — same boundary as credit-scoring above: GL Engine supplies the underlying trend data and event notifications, Scrip (or a dedicated analytics service) builds and owns the classification model.
 - Automated tax filing/remittance — FiSH computes and reports taxes due; submitting/paying that amount through each government's own e-filing or payment system is a later-phase integration, not v1.
 - Native mobile apps.
 - Full core-banking functions (card issuance, sort codes/account numbers) — these sit with the Banking-as-a-Service partner, not the GL Engine.
-

6. Target Users, Clients & Personas

- Purse member (UK/NI/ROI) — saves, borrows, views statements; interacts via Purse's app/web, not the GL Engine directly.
- Purse/Osusu member or savings-circle participant (Sierra Leone) — interacts via USSD/STK; same underlying ledger, different access channel.
- Purse operations/compliance staff — process loans, manage arrears stages, run transparency and regulatory reports; needs the Approver/Controller role.
- Bookkeeper/accountant (for general SMB/individual clients using FiSH GL independently of Purse) — day-to-day journal entries, period close.
- Mano River B2B company (Sierra Leone, Liberia, Guinea, Côte d'Ivoire) — an independent company that runs its bookkeeping on FiSH primarily to get an accurate ledger and, from it, its taxes due.
- Scrip (Rust) and Osusu (Kotlin) systems — post transactions and pull balances via API, not through the UI.
- Regulators/auditors (FCA, PRA, Bank of Sierra Leone, external ethics auditor per Purse's framework) — read-only access to audit trail and standing reports.
- Government / national tax authority (Mano River region) — recipient of the tax-due report a company generates from FiSH; not a direct FiSH user in v1, but the report's intended audience.

ClientType in the existing domain model (Individual, Sole Trader, Partnership, Company Limited, Non-Profit) maps to the general-purpose SMB use case; Purse itself, as a cooperative/credit union, doesn't yet map cleanly onto one of these (see Open Questions).

7. Functional Requirements

7.1 Chart of Accounts (COA)

- Company-level configurable COA with standard account types: Asset, Liability, Equity, Revenue, Expense.
- Default COA templates keyed to ClientType; a credit-union/cooperative template is needed for Purse (member shares/deposits, loan loss reserves, dividend payable, etc. — distinct from standard equity/share-capital structures).
- Parent/child account hierarchies and account codes; no hard deletes of accounts with historical activity.

7.2 Journal Entries & Double-Entry Bookkeeping

- All postings enforced as balanced double-entry, matching TransactionSide (Debit/Credit) and the accounting-equation rules already documented in code.
- JournalSource tagging on every entry (Manual, Reversal, System, API, Import, Integration, Closing) as already modelled.
- PostingStatus lifecycle exactly as coded: Draft ↔ Pending → Posted → Reversed, with Rejected as a return path from Draft/Pending, and System following Posted's rules. isEditable(), affectsBalance(), and isFinal() semantics should drive UI/API behaviour directly.
- Reversing and adjusting entries only — no edits to Posted/System entries.

7.3 Multi-Currency Support

- Per-company base/reporting currency; transaction-level currency with FX rate captured at entry time.
- Minimum currency set for v1: GBP, EUR, SLE, USD.
- Realised/unrealised FX gain-loss handling for period-end revaluation.

7.4 Accounting Periods & Close

- PeriodType options as coded: Month, Quarter, Year, Custom.
- PeriodStatus lifecycle exactly as coded: Draft → Open → Closed → Locked, with Closed → Open reopen support and Locked as terminal.
- Period-close checklist workflow (trial balance review, adjusting entries, lock) suited to both monthly SMB close and Purse's regulatory reporting calendar.

7.5 Audit Trail

- Every action logged against the existing AuditAction taxonomy: CRUD operations; state transitions (Posted, Reversed, Approved, Rejected, Submitted); period operations (Opened, Closed, Locked, Reopened); security events (Login, Logout, Login Failed, Password Changed,

Permission Granted/Revoked); data operations (Exported, Imported, Bulk Update); system operations (System Generated, Migration); access tracking (Viewed, Downloaded, Printed).

- Field-level change tracking for compliance (before/after values), matching the existing FieldChange model.
- No hard deletes of posted transactions — corrections via reversing entries only.
- Exportable audit log, using the existing ExportFormat options, for compliance/audit purposes (SOX/GDPR referenced explicitly in the current code comments; FCA/PRA and Bank of Sierra Leone reporting to be added).

7.6 Financial & Regulatory Reporting

- Standard statements: trial balance, income statement, balance sheet, cash flow statement, general ledger detail (by account/period, using existing OrderBy options).
- Multi-dimensional reporting using DimensionType tags (e.g., P&L by branch/region for Purse, by savings circle for Osusu).
- Purse-specific standing reports: lending by category vs. prohibited/grey-area list, loan-loss rate and causes, aggregate interest margin vs. cost, investment holdings disclosure — needed for the annual transparency report described in the ethical framework.
- Paginated API responses for all report/list endpoints, matching the existing PaginatedResult model.
- Export to CSV/PDF via the existing ExportFormat enum; report data also available via API.

7.7 Multi-Tenancy, Multi-User & Multi-Company Access

- Full data isolation between tenants.
- Multiple users per tenant, each with an assigned role; a user may belong to multiple tenants (e.g., a bookkeeper serving several clients).
- A tenant may represent a group of companies — e.g., Purse UK, Purse NI/ROI, and Purse Sierra Leone as separate companies under one tenant (or as separate tenants — see Open Questions), each with its own COA and books, supporting consolidated and per-entity reporting.
- Every operation carries a RequestContext (as already modelled) capturing who is acting, from where, for audit and security checks.

7.8 Roles & Permissions

- RBAC at minimum: Owner/Admin, Accountant, Approver, Read-only, plus a Compliance/Ethics-review role for Purse's board-review and grey-area escalation workflow.
- Permission checks enforced at the API layer, not just the UI.
- Invitation/onboarding flow for adding users to a tenant.

7.9 Product-Specific Lending Rules (Purse)

- Configurable interest-rate ceilings per loan product (Personal 12.68% APR, Emergency/Short-term 18% APR, Secured 9% APR, absolute ceiling 26.82% APR), enforced at posting time with the

full rate-breakdown recorded against the loan (cost of funds, operating costs, risk premium, capital reserve build, admin).

- Loan-purpose affirmation, not sector screening: at origination, the partner/borrower affirms (a recorded, audited declaration) that funds will not be used for illegal or unethical purposes. FiSH does not classify or screen business sectors — that requires expertise outside the ledger's scope. If a misrepresentation is later discovered, it's handled as a breach of the affirmation through the existing membership/discipline process (fraud or material misrepresentation), not a pre-emptive category judgment.
- Staged arrears workflow (Early Contact → Pastoral Engagement → Formal Review → Final Resolution) as trackable states on a loan, each transition audited, supporting outcomes including payment-plan restructure, payment holiday, partial write-off, and formal collection.
- “Year of Jubilee” debt-forgiveness as a supported transaction type (partial/full write-off against a documented hardship review), distinct from a standard write-off.

7.10 Integration & API Layer

- REST API (or gRPC, TBD) for posting journal entries, querying balances/reports, and managing COA/tenant/company settings — must interoperate cleanly with a Rust client (Scrip) and Kotlin clients (Osusu/Purse), so the protocol should stay language-agnostic.
- API authentication via tenant-scoped API keys or OAuth2, with rate limiting.
- Designed so a future USSD/STK gateway (Sierra Leone) and a future Banking-as-a-Service payments layer (UK) can post through the same interface without engine-level changes.
- Event notifications (webhook or message-queue publish) on journal-entry posting, so downstream consumers like Scrip's business-health monitoring (Section 7.14) can react close to the moment of posting rather than polling full reports on a fixed schedule.

7.11 Membership / Common-Bond Onboarding Support

- Client/member record must be able to capture Purse's “common bond” declaration (Christian faith affirmation, Apostles'/Nicene Creed) as onboarding metadata, alongside standard KYC fields (ID, proof of address, source of funds) — the GL Engine records the outcome and gates postings, without performing the verification itself (see Section 5.2, Out of Scope).

7.12 Tax Computation & Government Reporting

- Configurable tax rules per jurisdiction, starting with the Mano River launch markets (Sierra Leone, Liberia, Guinea, Côte d'Ivoire): tax type (e.g., corporate income tax, VAT/GST, payroll/PAYE where applicable), rate, and which accounts/periods it applies to — stored as data, not hard-coded, since rates and rules change by country and over time.
- Tax-due computation derived directly from a company's posted (PostingStatus.POSTED/SYSTEM) ledger transactions for a given period — no separate tax subledger required for v1; reuses the existing Account, Period, and DimensionType model.
- A generated tax report per company/period, in a format suitable for a business to submit to its national tax authority, distinguishing what's computed/reported (a FiSH responsibility) from what's remitted/paid (the company's own responsibility, or a later-phase government e-filing integration — see Section 5.2).

- Available to any FiSH client — Purse companies, Osusu/Scrip entities, general SMB clients, and Mano River B2B companies alike — not a Purse-specific feature.

7.13 Company Debt Restructuring (Capitalization & Turnaround)

- For company/business borrowers specifically (not individual members, who remain under the Jubilee write-off model in 7.9), the arrears Final Resolution stage supports a third outcome alongside partial write-off and formal collection: debt capitalization — converting the outstanding loan balance into an equity stake in the borrowing company.
- Capitalization is a distinct, audited transaction type: the loan receivable is derecognized and an equity/investment holding is recognized in its place, at an agreed valuation — not a routine write-off.
- The resulting position is tracked as a distinct EquityStake entity linked back to the originating loan/arrears case, capturing conversion date, converted amount, resulting ownership percentage, valuation basis, and status (Under Administration, Turnaround In Progress, Exited).
- Business continuity is the explicit objective of this workflow, not maximising recovery — the record should support tracking turnaround plan/administration activity, not just the financial position.
- Because active investment/turnaround management sits outside Purse's day-to-day credit-union ledger function, an EquityStake is expected to transfer to or be co-managed with Scrip (Section 11.1); the GL Engine should support attributing or moving the position to a Scrip-managed company/tenant.
- Exit is a distinct, board-approved event, not an assumed default outcome: (a) sale of the stake back to the original owner if turnaround circumstances make that possible, (b) sale to a third party, or (c) write-off if turnaround is not achieved — each its own auditable transaction type.

7.14 Business Health Signal (feeds Scrip's Treasury Management Services)

- GL Engine's responsibility is limited to supplying the underlying data: near-real-time access to a company's posted transactions and standard trend metrics (revenue/expense trajectory, cash flow trend over rolling periods) via the API and event notifications in Section 7.10 — reusing the existing Account, Period, and DimensionType model, no new ledger entity required.
- The actual classification logic that turns this trend data into a Growing / Stagnant / Regressing / Dying signal is out of scope for the GL Engine (Section 5.2) — that belongs to Scrip or a dedicated analytics service, consistent with the same boundary already set for Osusu's credit-scoring engine.
- “Real time” in a ledger-backed system realistically means as close to the moment of posting as the event-notification architecture allows, not a live market-data feed — this should be stated explicitly to set expectations with Scrip's product design.
- This signal is a natural early-warning input to the arrears/turnaround workflow (Section 7.13): a company trending toward “Regressing” or “Dying” is a candidate for proactive engagement, though whether that link is automatic or advisory-only is an open question (Section 13).

8. Non-Functional Requirements (SaaS Architecture)

8.1 Multi-Tenancy Architecture

- Strict logical (and/or physical, TBD during architecture) data isolation between tenants.
- Tenant identification enforced at every data access layer.

8.2 Security & Compliance

- Encryption in transit (TLS) and at rest (AES-256, matching Purse's stated technology architecture).
- ISO 27001-aligned information security practices.
- Role-based access control enforced server-side.
- Audit logging of authentication/authorization events, in addition to the financial audit trail.
- PCI-DSS alignment if/when the engine touches card-related data (likely handled by the Banking-as-a-Service partner instead, per Section 5.2).
- Data sovereignty consideration: Purse's stated infrastructure target is AWS UK regions for UK data; Sierra Leone data residency requirements to be confirmed with Bank of Sierra Leone Sandbox guidance.
- Compliance posture targets: FCA/PRA (UK), GDPR/ICO (UK/EU), Bank of Sierra Leone Sandbox requirements (SL), plus general SOX/GDPR-style audit trail already referenced in code comments.

8.3 Scalability & Performance

- Architecture should scale horizontally as tenant count and transaction volume grow.
- Report generation should perform acceptably for tenants with multiple years of transaction history.

8.4 Availability & Reliability

- Target Recovery Time Objective (RTO) and Recovery Point Objective (RPO) should align with Purse's stated business-continuity targets (4-hour RTO, 1-hour RPO) once the GL Engine is in the critical path for live transactions.
- Regular automated backups with tested restore procedures; real-time replication to a secondary region as the platform matures.

8.5 Data Retention & Backup

- Financial records retained indefinitely by default (configurable per regulatory needs); AML/KYC-adjacent records retained per the 5-year CDD retention period referenced in Purse's plan, where the GL Engine stores related metadata.
- Point-in-time recovery capability for the ledger database.

9. Existing Implementation Status

As of 2026-08-11, the FiSH repository already contains Kotlin domain types under `com.theprodeogroup.fish.domain.common`, which this specification treats as the current source of truth for the enums/state machines described above:

File	Contents
<code>audit_action.kt</code>	AuditAction enum — CRUD, state transitions, period ops, security events, data ops, system ops, access tracking
<code>client_type.kt</code>	ClientType enum — Individual, Sole Trader, Partnership, Company Limited, Non-Profit
<code>posting_status.kt</code>	PostingStatus enum + transition rules — Draft, Pending, Posted, Reversed, Rejected, System
<code>period_status.kt</code>	PeriodStatus enum + transition rules — Draft, Open, Closed, Locked
<code>period_type.kt</code>	PeriodType enum — Month, Quarter, Year, Custom
<code>journal_source.kt</code>	JournalSource enum — Manual, Reversal, System, API, Import, Integration, Closing
<code>transaction_side.kt</code>	TransactionSide enum — Debit, Credit
<code>dimension_type.kt</code>	DimensionType enum — Cost Center, Department, Project, Location, Product Line, Customer, Vendor, 3 custom slots
<code>field_change.kt</code>	Field-level change record for audit trail
<code>export_format.kt</code>	Supported export formats
<code>order_by.kt</code>	Query ordering options
<code>paginated_result.kt</code>	Generic paginated API response wrapper
<code>request_context.kt</code>	Who/where context for audit and security checks
<code>validation_result.kt</code>	Validation pass/fail + errors/warnings

This spec has been written to be consistent with this existing code rather than propose a competing model.

10. High-Level Data Model (Illustrative)

Entity	Description
Tenant	An organization/customer account; owns all other tenant-scoped data. May represent Purse UK, Purse SL, or an independent SMB client.
Company	A legal entity/company within a Tenant; a Tenant may contain multiple Companies (group structure — e.g., Purse's UK/NI/ROI/SL entities).
Client	A ledger client of a given ClientType (Individual, Sole Trader, Partnership, Company Limited, Non-Profit); credit-union/cooperative type TBD.
User	A person who can log in; may belong to one or more Tenants via Membership.
Membership	Join of User + Tenant + Role, defining a user's access within a tenant.
Account (COA)	A ledger account belonging to a Company, with type, code, and hierarchy.
JournalEntry	A balanced set of debit/credit lines, with PostingStatus, JournalSource, and audit metadata.
JournalLine	A single debit or credit line within a JournalEntry, referencing an Account and optionally tagged with DimensionType values.

Entity	Description
Period	A fiscal period belonging to a Company, with PeriodType and PeriodStatus.
Currency / FXRate	Supported currencies (GBP, EUR, SLE, USD at minimum) and exchange rates for multi-currency postings.
AuditLogEntry	Immutable record of an AuditAction, with FieldChange details where applicable.
APIKey	Tenant-scoped credential for programmatic access (Scrip, Osusu, future USSD gateway).
LoanProduct	Purse-specific: product type, APR ceiling, rate-breakdown components.
ArrearsCase	Purse-specific: staged arrears record linked to a loan, tracking stage transitions and outcomes.
TaxRule	Jurisdiction-specific tax configuration: tax type, rate, applicable accounts/periods.
TaxComputation	Computed tax-due record for a Company/Period, derived from posted ledger data, feeding the government-ready tax report.
EquityStake	An ownership position in a company, typically arising from debt capitalization (Section 7.13); tracks conversion terms, valuation, administration/turnaround status, and eventual exit — expected to be Scrip-managed.

This model is illustrative and will be refined during technical design, in step with the existing Kotlin code.

11. Relationship to Other FiSH Components

11.1 Scrip (Treasury / Investment Arm)

Being written in Rust, kept as a legally separate entity from Purse. Expected to post treasury-related transactions (pooled deposits, Gilts/bonds, gold-referenced reserve activity) into the GL Engine via the API layer, using it as the system of record for financial reporting. Since Scrip is a Rust client, the API layer (Section 7.10) must stay language-agnostic rather than assume a JVM-based client. Scrip's role also extends to workout/restructuring positions: EquityStakes arising from Purse's company debt-capitalization process (Section 7.13) are expected to transfer to or be co-managed with Scrip, since active turnaround/administration management is an investment function, not a credit-union lending function.

11.2 Osusu (run through Purse)

Being written in Kotlin. Posts member contributions, payouts, and Osusu Advance micro-lending activity into the GL Engine. In the UK, Osusu sits inside Purse's broader product set; in Sierra Leone, it is the primary product, accessed via USSD/STK rather than a smartphone app — both channels should post through the same GL Engine API.

11.3 Purse (Credit Union)

Purse is the primary consumer of the GL Engine for UK/NI/ROI operations, and (via the Sierra Leone plan) for SL operations as well. Purse's ethical/theological lending framework (Section 2.5) translates

directly into GL Engine requirements: rate-ceiling enforcement, prohibited-purpose flagging, staged arrears tracking, and Jubilee forgiveness as a first-class transaction type.

12. Assumptions

- The GL Engine continues to be built to also serve general SMB/individual clients (per the existing ClientType model), not exclusively Purse/Scrip/Osusu.
 - Core tech stack is Kotlin/Java (GL Engine, Osusu, Purse-related code), Rust (Scrip), PostgreSQL, AWS UK regions — as stated in Purse's Business Plan and reflected in the existing repository.
 - Compliance targets include FCA/PRA/FSCS/FOS/GDPR-ICO (UK), Bank of Sierra Leone Sandbox/SACCO requirements (SL), and general SOX/GDPR-style audit-trail practice already referenced in code comments.
 - AML/KYC screening, payment rails, and USSD gateway are external systems the GL Engine integrates with, not systems it builds.
 - FiSH computes and reports taxes due but does not remit payment to government or file directly with a tax authority in v1, unless a specific e-filing integration is later scoped.
-

13. Open Questions

- What ledger ClientType (or new type) represents Purse itself, given it is a cooperative/credit union, not Individual/Sole Trader/Partnership/Company Limited/Non-Profit as currently modelled?
- Should Purse UK, Purse NI/ROI, and Purse Sierra Leone be modelled as one Tenant with multiple Companies, or as separate Tenants? (Different regulators, currencies, and — for SL — a different access channel argue for at least separate Companies; the degree of shared reporting needed will decide Tenant vs. Company.)
- What protocol will the GL Engine's API use so it interoperates cleanly with a Rust client (Scrip) and Kotlin clients (Osusu/Purse) — REST/JSON, gRPC, or another option?
- How much of the AML/KYC outcome (PEP screening result, common-bond verification) does the GL Engine need to store as metadata vs. simply gate on a boolean/status flag from an external system?
- What's the v1 boundary for the interest-rate-ceiling and arrears-workflow features — full enforcement, or just the data model to support them once Purse's lending product is live?
- Does the credit-scoring engine (Osusu) need a formal read API into transaction history, or is it expected to consume the same reporting API as everything else?
- What does Sierra Leone's Bank of Sierra Leone Sandbox process require in terms of data residency or reporting format that might differ from the UK/EU requirements?
- Which specific taxes must FiSH compute first for the Mano River launch markets — corporate income tax, VAT/GST, payroll/PAYE, or some combination, and does this differ by country?

- Does any Mano River tax authority currently offer an e-filing API FiSH could integrate with in a later phase, or is manual submission of a FiSH-generated report the realistic v1 expectation?
 - Who is the paying customer for the B2B/Mano River ledger offering — the company itself (SaaS subscription), or could a government/tax authority also be a counterparty (e.g., a compliance-data arrangement)?
 - Does Purse or Scrip legally hold EquityStakes arising from debt capitalization, and does the GL Engine need to model a formal transfer/hand-off between them, or just tag the position as Scrip-managed from creation?
 - What governance/valuation process sets the capitalization terms (how much debt converts to what ownership percentage), and does that decision need to be captured as its own auditable event before the capitalization posts?
 - What criteria distinguish a “turnaround” candidate (capitalization) from a hardship write-off (Jubilee) or formal collection — a board judgment call captured narratively, or something the system should structure/prompt for?
 - What specific metrics and thresholds define Growing/Stagnant/Regressing/Dying, and who owns that definition — Scrip product design, or does FiSH need to expose configurable metric definitions per company/sector?
 - What latency counts as “real time” for the business-health signal — event-driven notification within seconds/minutes of posting, or a defined acceptable delay?
 - Should a “Regressing” or “Dying” signal automatically trigger the arrears/turnaround workflow (Section 7.13) for early intervention, or remain a separate advisory signal Scrip surfaces to the owner without an automatic lending-side consequence?
-

14. Glossary

- COA — Chart of Accounts.
- GL — General Ledger.
- RBAC — Role-Based Access Control.
- Tenant — A customer organization using the SaaS product, with isolated data.
- FX — Foreign exchange.
- SaaS — Software as a Service.
- FCA / PRA — UK Financial Conduct Authority / Prudential Regulation Authority.
- FSCS / FOS — Financial Services Compensation Scheme / Financial Ombudsman Service (UK).
- BSL — Bank of Sierra Leone.
- SACCO — Savings and Credit Co-Operative (Sierra Leone Cooperative Societies Act 1977).
- KYC / AML / PEP / SAR — Know Your Customer / Anti-Money Laundering / Politically Exposed Person / Suspicious Activity Report.
- Common bond — The shared-membership criterion (here, Christian faith affirmation) required for credit union membership.